Requirements Traceability Matrix (RTM)

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Purpose: This matrix demonstrates that every Functional Requirement (FR) is traced forward to the design artefacts that implement it, the source code that realises it, and the automated test(s) that verify it. It also demonstrates backward traceability from tests to requirements, ensuring no orphan tests exist.

Legend:

- **Design:** UML artefacts that model the requirement (UC = Use Case, SQ = Sequence, CL = Class).
- **Code:** File(s) in the source tree that implement the requirement.
- **Test:** Automated tests in `tests/` that verify the requirement.

---

1. Functional Requirements Traceability

Req ID	Requirement (short)	Design Artefact	Implementation	Test(s)
FR-01	Register with email/password/name/role/matric	UC-01, SQ-Login (partial)	`app/routers/auth.py` (`/auth/register`), `app/services/user_service.py` (`create_user`), `app/schemas/user.py` (`UserCreate`)	`test_api_integration.py::test_register_returns_201_and_us`
FR-02	Reject duplicate email	UC-01	`app/services/user_service.py` (`create_user` raises `ValueError`)	`test_api_integration.py::test_register_duplicate_email_re` `test_user_service.py::test_create_user_duplicate_email_ra`
FR-03	Hash passwords with bcrypt	CL-User	`app/utils/security.py` (`hash_password`)	`test_security.py::test_hash_password_returns_string`, `test_hash_password_is_not_plaintext`, `test_hash_password_sa` `test_user_service.py::test_create_user_hashes_password`
FR-04	Login with email + password	UC-02, SQ-Login	`app/routers/auth.py` (`/auth/login`), `app/services/user_service.py` (`authenticate_user`)	`test_api_integration.py::test_login_returns_token`, `test_login_wrong_password_returns_401`, `test_login_unknown_` `test_user_service.py::test_authenticate_user_correct_pass` `test_authenticate_user_wrong_password`, `test_authenticate_u`
FR-05	Issue signed JWT (15 min exp)	SQ-Login	`app/utils/jwt.py` (`create_access_token`), `app/config.py`	`test_api_integration.py::test_login_returns_token`
FR-06	Retrieve own profile via `/auth/me`	UC-03, SQ-Login	`app/routers/auth.py` (`/auth/me`), `app/utils/jwt.py` (`get_current_user`)	`test_api_integration.py::test_me_with_valid_token_returns` `test_me_without_token_returns_401`, `test_me_with_bad_token_`
FR-07	Generate QR digital ID card	UC-04, CL-User	`app/routers/id_card.py`, `app/services/qr_service.py`	`test_api_integration.py::test_id_card_returns_qr_data_url` `test_id_card_requires_auth`
FR-08	Auto-create wallet	UC-05, CL-Wallet	`app/services/wallet_service.py` (`get_or_create_wallet`)	`test_wallet_service.py::test_get_or_create_wallet_creates` `test_get_or_create_wallet_is_idempotent`, `test_api_integration.py::test_wallet_auto_created_with_ze`
FR-09	Top up wallet (positive amount)	UC-05, SQ-Wallet	`app/routers/wallet.py` (`/wallet/topup`), `app/services/wallet_service.py` (`top_up`)	`test_wallet_service.py::test_top_up_increases_balance`, `test_top_up_multiple_times_accumulates`, `test_top_up_zero_r` `test_top_up_negative_raises`, `test_api_integration.py::test_wallet_topup_credits_balanc` `test_wallet_negative_topup_returns_422`
FR-10	Pay from wallet (sufficient balance)	UC-06, SQ-Wallet	`app/routers/wallet.py` (`/wallet/pay`), `app/services/wallet_service.py` (`pay`)	`test_wallet_service.py::test_pay_decreases_balance`, `test_pay_insufficient_funds_raises`, `test_pay_insufficient_funds_does_not_change_balance`, `test_` `test_api_integration.py::test_wallet_pay_debits_balance`, `test_wallet_pay_insufficient_funds_returns_400`
FR-11	Record transaction with timestamp	CL-Transaction	`app/models/wallet.py` (`Transaction` model)	`test_wallet_service.py::test_top_up_increases_balance`, `tes`
FR-12	View transaction history	UC-07, SQ-Wallet	`app/routers/wallet.py` (`/wallet/transactions`), `app/services/wallet_service.py` (`list_transactions`)	`test_wallet_service.py::test_list_transactions_returns_al` `test_list_transactions_newest_first`, `test_api_integration.py::test_wallet_transactions_returns`
FR-13	Role distinction (student/staff/admin)	CL-User, UC-09	`app/models/user.py` (`role` column), `app/schemas/user.py`	`test_user_service.py::test_create_user_returns_user` (check

Req ID	Requirement (short)	Design Artefact	Implementation	Test(s)
FR-14	Health-check endpoint	—	`app/routers/health.py`	`test_api_integration.py::test_health_endpoint`
FR-15	Web UI login + dashboard	UC-01, UC-02, UC-03, UC-07	`app/routers/pages.py`, `app/templates/login.html`, `app/templates/dashboard.html`, `app/static/`	*(Manual/visual verification; automated UI tests not in scope)*

2. Non-Functional Requirements Traceability

Req ID	NFR	Design / Config	Verification
NFR-01	p95 login ≤ 500 ms	FastAPI async, indexed email lookup (`app/models/user.py`)	Manual timing; benchmarkable with `wrk` or `locust`
NFR-02	bcrypt cost ≥ 12	`app/utils/security.py` (`bcrypt.gensalt()`)	`test_security.py` suite
NFR-03	Reject unauthorized requests	`app/utils/jwt.py` (`get_current_user`)	`test_api_integration.py::test_me_without_token_returns_401`, `test_me_with_bad_token_returns_401`, `test_id_card_requires_auth`, `test_wallet_requires_auth`
NFR-04	JWT HS256, ≥ 32-byte secret	`app/config.py` (`jwt_secret`, `jwt_algorithm`)	PyJWT warning-free run (verified after key-length fix)
NFR-05	`/health` returns 200	`app/routers/health.py`	`test_api_integration.py::test_health_endpoint`
NFR-06	Register + login ≤ 2 min	UX design in `login.html` / `dashboard.html`	Manual usability check
NFR-07	Modular maintainability	Router/service/model/schema layering across `app/`	Code review; low cyclomatic complexity per function
NFR-08	≥ 70 % coverage	pytest + coverage plugin	`pytest --cov=app --cov-report=term-missing`
NFR-09	Windows + Linux portable	Python 3.12, no OS-specific code	GitHub Actions CI on Ubuntu + local Windows runs
NFR-10	OpenAPI 3.0 docs	FastAPI auto-generated `/docs`	Browser verification; used for demo
NFR-11	Stateless scale-out	JWT (no session state)	Architectural design
NFR-12	99 % uptime target	Single-node deployment; UptimeRobot on `/health`	Manual monitoring during exam period

3. Coverage Summary

Metric	Value
Total FRs	15
FRs with automated tests	14 (93 %)
FRs verified manually (UI)	1 (FR-15)
Total NFRs	12
NFRs with automated verification	5 (NFR-02, 03, 04, 05, 08)
NFRs verified by design/manual	7
Total automated tests	53
Tests passing	53 (100 %)
CI status	Green ✅

4. Backward Traceability — Sample

To demonstrate backward traceability, here are three tests and the requirement(s) they verify:

Test	Verifies
`test_wallet_pay_insufficient_funds_does_not_change_balance`	FR-10 (Pay from wallet) + NFR-04 (fail-secure balance)
`test_register_duplicate_email_returns_400`	FR-02 (Duplicate email)
`test_hash_password_same_input_different_output`	FR-03 (bcrypt hashing) + NFR-02 (cost)

---

5. Change Impact Analysis (Example)

If **FR-10 (Pay from wallet)** were modified (e.g., adding a service fee), the following artefacts would need review:

1. `app/services/wallet_service.py` → `pay()` function
2. `app/routers/wallet.py` → `/wallet/pay` endpoint
3. `app/schemas/wallet.py` → `PayRequest` schema
4. Tests: all `test_pay_*` and `test_wallet_pay_*` tests
5. Documentation: SRS Section 3.1, this RTM, user manual

This is why traceability matters — a single requirement change cascades predictably.

End of Requirements Traceability Matrix.